

Playwright Automation Framework – Nykaa website

1. Project Title

UI Automation Testing Framework using Playwright – Search Functionality Validation

Github url: [ust-sdet-training/SDET at maniprasad-24jun26](https://github.com/maniprasad24jun26/ust-sdet-training/SDET)

2. Objective

The objective of this project is to design and implement a scalable UI automation framework using Playwright and TypeScript to validate search functionality for a web application.

Target Website:

[Buy Cosmetics Products & Beauty Products Online in India at Best Price | Nykaa](https://www.nykaa.com/)

3. User Story

User Story 1 – Existing Product Search

As a user,
I want to search for a product,
So that I can see all the products available.

Acceptance Criteria:

- User opens the Nykaa website
 - User searches for a product
 - User clicks enter
 - User is redirected to product page
-

User Story 2 – Existing Product Search

Acceptance Criteria:

- No results found message is displayed
-

4. Environment & Setup

Hardware

- System: Windows
- RAM: Minimum 8 GB
- Processor: Intel i5 or above

Software

Component Version

Node.js Latest LTS

Playwright Latest

TypeScript Latest

VS Code Latest

Browser Chromium

5. Framework Design

Framework Architecture:

Fixture Layer

↓

Flow Layer

↓

Page Layer

↓

Test Layer

Description:

Fixtures

Responsible for:

- Creating reusable test context
- Injecting page objects

File:

fixtures/fixture.ts

Flow Layer

Responsible for:

- Business workflow implementation
- Combining page actions

File:

flows/TestFlow.ts

Methods:

- positiveTest()
 - negativeTest()
-

Page Layer

Responsible for:

- Encapsulating locators
- Maintaining UI interactions

Files:

pages/HomePage.ts

pages/ProductsPage.ts

Test Layer

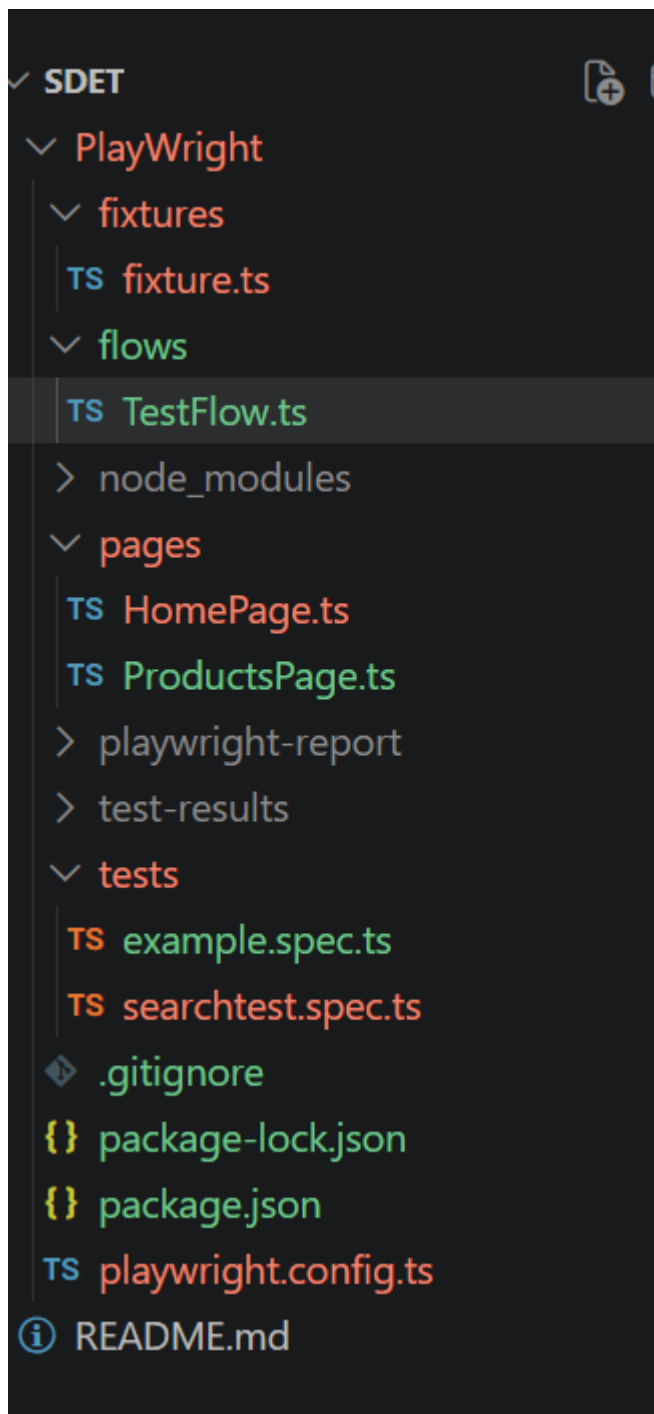
Responsible for:

- Test orchestration
- Assertions

File:

tests/searchtest.spec.ts

6. Project Structure



7. Scenarios Covered

Scenario 1 – Searching for existing product

Input:

Search: “Cosmetics”

Expected:

- Redirected products page

- Available products are visible

Status:

PASS

Scenario 2 – Searching for non-existing product

Input:

Search: “Mobikes”

Expected:

- No results found is displayed

Status:

PASS

8. Execution Steps

Install dependencies:

`npm install`

Install browsers:

`npx playwright install`

Execute tests:

`npx playwright test tests/searchtest.spec.ts --headed`

Open report:

`npx playwright show-report`

9. Test Results

Execution Summary:

Total Tests: 2

Passed: 2

Failed: 0

Execution Mode:

Chromium Browser

Execution Duration:

~14 seconds

10. Evidence

Execution Output:

Running 2 tests using 1 worker

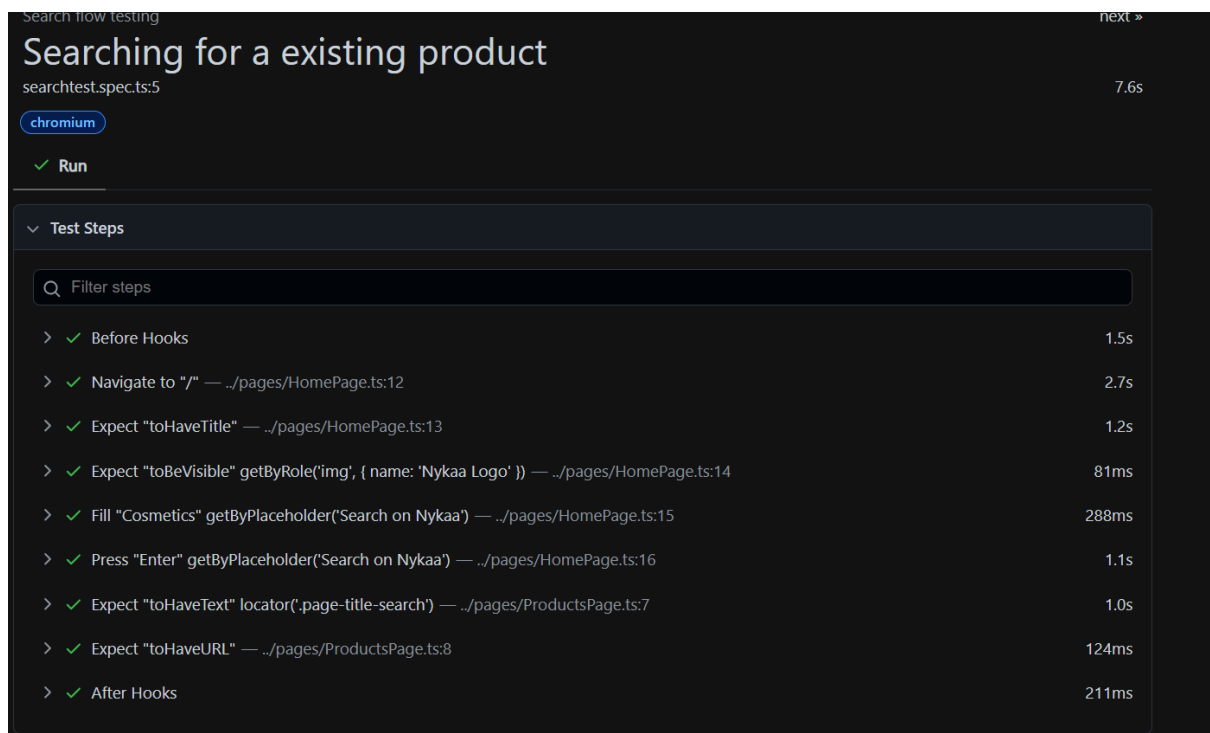
✓ Search existing product

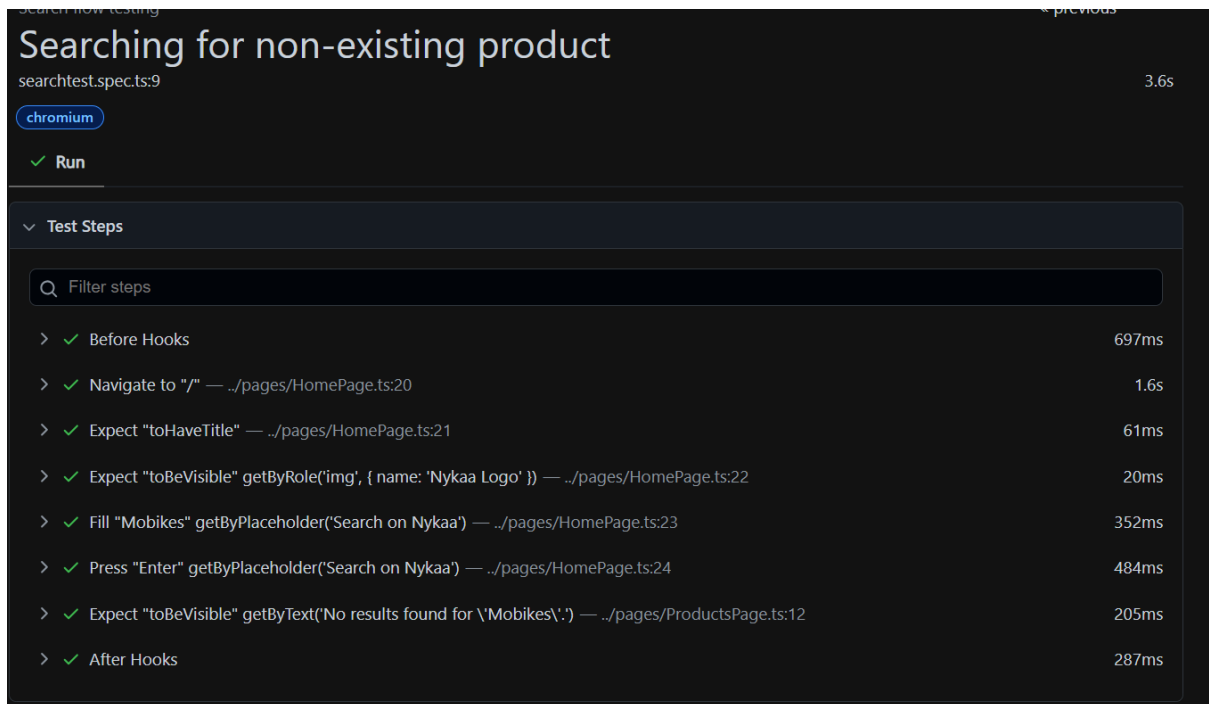
✓ Search non-existing product

2 passed

HTML Report Generated Successfully

Screenshots:





Trace:

Enabled on first retry

11. Environment Configuration

Playwright Configuration:

- Browser: Chromium
- Trace Collection: Enabled
- Base URL Configured
- HTML Reporting Enabled

Configuration Parameters:

- retries
 - workers
 - trace
 - baseURL
-

12. Issues Encountered

Issue 1:

ERR_ABORTED – Frame Detached

Cause:

Asynchronous execution without await

Resolution:

Implemented await for flow methods

Issue 2:

Navigation Timeout

Cause:

External website instability

Resolution:

Used optimized navigation strategy

Issue 3:

Incorrect Locator Strategy

Cause:

Invalid role and label selection

Resolution:

Updated locator methods

13. Limitations

- Dependent on external website availability
 - Execution affected by network speed
 - Limited browser coverage
 - Public site introduces flakiness
-

14. Conclusion

The Playwright automation framework was successfully implemented using Page Object Model, Flow Layer, and Fixtures.

The framework achieved:

- Reusable architecture
- Improved maintainability

- Reduced code duplication
- Automated validation of search functionality
- Execution reporting and evidence generation